

TCCL: Discovering Better Communication Paths for PCIe GPU Clusters

Heehoon Kim
heehoon@aces.snu.ac.kr
Dept. of Computer Science and
Engineering
Seoul National University
Seoul, South Korea

Junyeol Ryu
junyeol@aces.snu.ac.kr
Dept. of Computer Science and
Engineering
Seoul National University
Seoul, South Korea

Jaejin Lee
jaejin@snu.ac.kr
Dept. of Data Science
Dept. of Computer Science and
Engineering
Seoul National University
Seoul, South Korea

Abstract

Exploiting parallelism to train deep learning models requires GPUs to cooperate through collective communication primitives. While systems like DGX, equipped with proprietary interconnects, have been extensively studied, the systems where GPUs mainly communicate through PCIe have received limited attention. This paper introduces TCCL, a collective communication library designed explicitly for such systems. TCCL has three components: a profiler for multi-transfer performance measurement, a pathfinder to discover optimal communication paths, and a modified runtime of NCCL to utilize the identified paths. The focus is on ring-based collective communication algorithms that apply to popular communication operations in deep learning, such as AllReduce and AllGather. The evaluation results of TCCL on three different PCIe-dependent GPU clusters show that TCCL outperforms (up to $\times 2.07$) the state-of-the-art communication libraries, NCCL and MSCCL. We also evaluate TCCL with DL training workloads with various combinations of parallelism types.

CCS Concepts: • Software and its engineering → Cooperating communicating processes; • Computer systems organization → Interconnection architectures.

Keywords: Cluster, GPU, Collective communication, PCIe

ACM Reference Format:

Heehoon Kim, Junyeol Ryu, and Jaejin Lee. 2024. TCCL: Discovering Better Communication Paths for PCIe GPU Clusters. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '24)*, April 27–May 1, 2024, La Jolla, CA, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3620666.3651362>



This work is licensed under a Creative Commons Attribution International 4.0 License.

ASPLOS '24, April 27–May 1, 2024, La Jolla, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0386-7/24/04.

<https://doi.org/10.1145/3620666.3651362>

1 Introduction

Over the last few years, the number of parameters in the state-of-the-art deep learning (DL) models has grown to trillions [8]. To train such models, a common practice is to leverage parallelism [13, 52, 59]; dividing computation and memory requirements into multiple GPUs. Parallel training introduces GPU communication to share computation results or synchronize training states. Hence, the performance of GPU communication has become critical to training throughput in that when communication lags behind computation speed, GPUs become underutilized or idle, waiting for data [10, 57].

As a response, GPU vendors provide collective communication libraries specialized for their GPU products. For example, NVIDIA, Intel, and AMD offer NCCL, oneCCL, and RCCL, respectively [17, 20, 40]. Many studies build upon such vendor-provided libraries to find more efficient communication schedules [3, 58, 64] or communicate through paths with less congestion [29, 61]. Other studies propose frameworks to jointly optimize communication with computation involved in DL model training. They exploit model architectures to fuse communication and computation kernels [23, 48, 62] or find an execution schedule that minimizes interference between communication and computation kernels [30, 56].

We witness that almost all previous GPU communication library studies evaluate their proposals on the whole off-the-shelf systems produced by the GPU vendors or the vendors' partners, such as NVIDIA DGX-1 shown in Figure 1a. We refer to such GPU systems as *high-bandwidth systems*, considering the following distinguishable features:

- GPUs are connected by proprietary high-speed interconnect, such as NVLink [44] and NVSwitch [38].
- To maximize the number of PCIe devices in the system, PCIe devices are attached to external PCIe switches. These switches are non-blocking, i.e., they can handle all ports simultaneously at full rate.

However, many academic institutions and startups still rely on cost-efficient systems built by assembling off-the-shelf server components and GPUs, as shown in Figure 1b. We refer to these systems as *PCIe-dependent systems* because

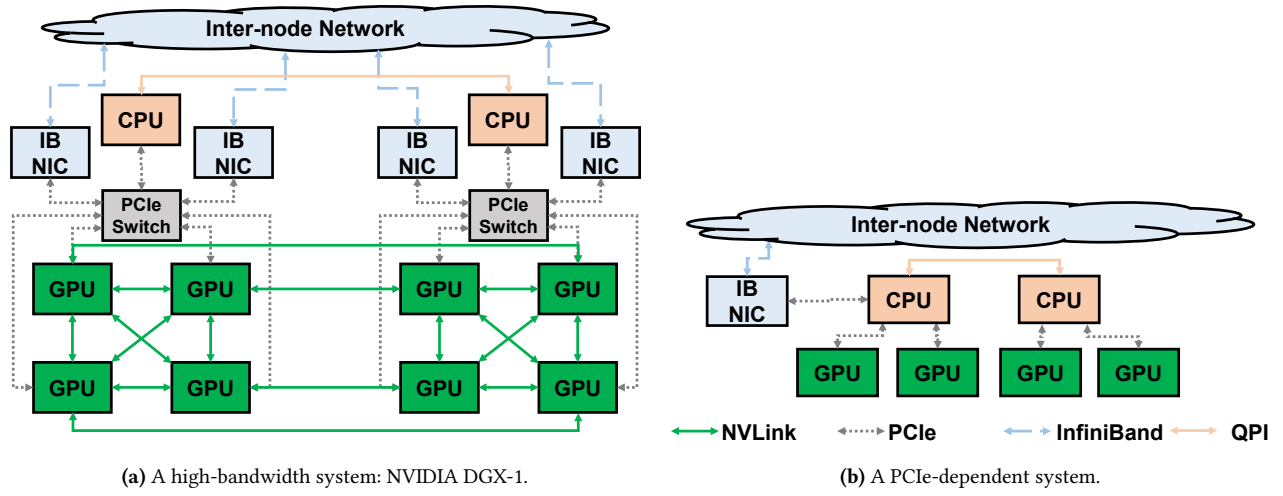


Figure 1. Topologies of a high-bandwidth and a PCIe-dependent systems.

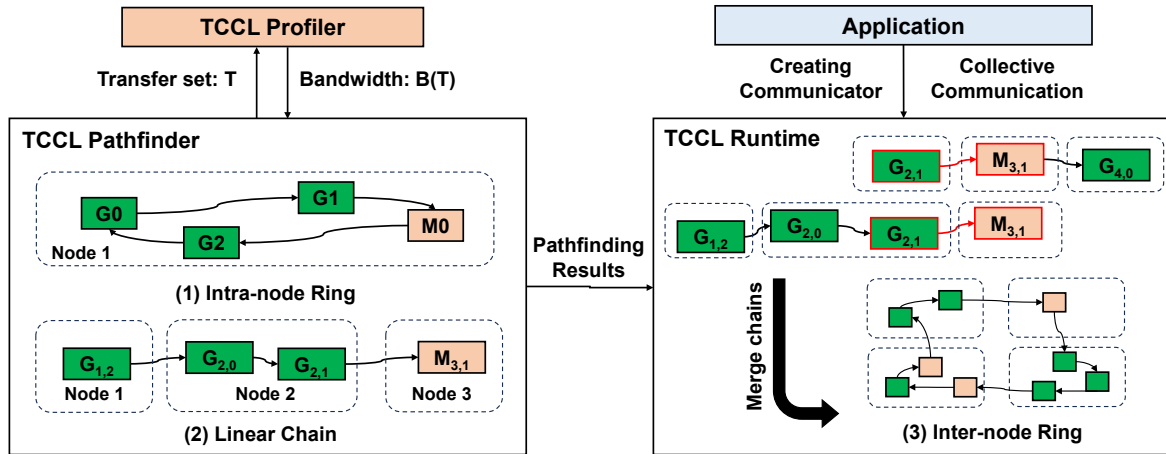


Figure 2. Overview of TCCL.

GPUs must communicate through the PCIe host bridge of the CPU and possibly the interconnect between NUMA nodes or CPU sockets. Cloud providers' virtual machines also include a catalog of PCIe-dependent systems [4, 11, 18, 32].

Based on the existing analysis work on GPU cluster traces such as Microsoft's Philly trace [24] and Alibaba's PAI trace [63], PCIe-dependent systems cover 90.4% of the machine specs that appear in the traces. Thus, GPU collective communication in PCIe-dependent systems is understudied compared to their popularity.

Surprisingly, existing collective communication libraries do not achieve the target performance that their performance model estimates in PCIe-dependent systems. One of the reasons is that transfers slow down when multiple transfers go

through the CPU in a specific pattern due to NUMA architectures. Another is the poor placement of the bounce buffers when GPUs communicate via CPU memory. Such congestion patterns are not identified or avoided by the existing collective communication libraries because the phenomenon does not appear when communication utilizes links outside the CPU, as in high-bandwidth systems.

To this end, we propose TCCL, a collective communication library that maximizes collective communication performance in PCIe-dependent systems. TCCL uses several methods to alter communication paths to avoid congestion. One is exploiting a bounce buffer on the CPU memory when transferring data between GPUs and NICs. While direct communication alone may have smaller latency and higher bandwidth, it may trigger congestion when used in collective

communication where multiple transfers are in progress simultaneously. Another is changing the virtual topology on which the collective communication algorithm operates. For example, when a ring algorithm is used, the order of GPUs in a ring can be altered to avoid congestion.

Figure 2 illustrates the overview of TCCL. TCCL consists of three components: profiler, pathfinder, and runtime. Since existing libraries [2, 41, 46] measure only the performance of a single transfer, we build a profiler that performs multiple transfers simultaneously and measures the time when all the transfers finish. The pathfinder is a module that searches for communication paths. The pathfinder always profiles each communication path of interest to check the performance it achieves to avoid congestion by taking it. Since the number of possible communication paths increases exponentially with the number of GPUs in a cluster, it is infeasible to brute-force search all the communication paths. Thus, the pathfinder first finds intra-node paths that visit only the GPUs in a single node. Then, it concatenates intra-node paths to construct inter-node paths using dynamic programming. We employ the Dijkstra-like search algorithm to reduce the search space further and consider NUMA symmetry. The runtime is a modification of NCCL such that the paths collective communication operations use are replaced with the paths found by the pathfinder while allowing a transparent interface to use TCCL without any code changes.

TCCL currently focuses on collective communication based on the ring algorithm, i.e., TCCL optimizes the communication paths that form a ring. The ring algorithm applies to various types of collective communication, including AllReduce, AllGather, and ReduceScatter, which are the most frequently used operations in DL. Also, the primary purpose of this work is to show the importance of avoiding congestion to improve the training throughput. Nevertheless, the methodology is applicable to other algorithms used by collective communication.

The major contributions of this paper are summarized as follows:

- To our knowledge, we are the first to uncover the communication patterns affected by congestion in PCIe-dependent systems.
- We develop TCCL, a collective communication library that automatically finds performant communication paths for ring-based collective communication operations in PCIe-dependent systems. TCCL is implemented on top of NCCL, so the existing software built with NCCL, such as PyTorch, can easily use TCCL.
- We evaluate TCCL on three different PCIe-dependent GPU clusters and TCCL outperforms (up to $\times 2.07$) the state-of-the-art communication libraries, NCCL and MSCCL.

2 Background

In this section, we briefly overview the low-level communication methods that involve GPUs and collective communication between them.

2.1 Communication Mechanisms for GPUs

There are two primary ways for GPUs to transfer data from/to the CPU's main memory: *CPU-initiated transfer* and *GPU-initiated transfer*. The CPU-initiated transfer exploits DMA. Since modern GPUs have a DMA engine, the CPU programs the DMA engine using API calls, such as `cudaMemcpy`, and triggers the DMA data transfer. On the other hand, the GPU-initiated transfer maps a memory space in the CPU's address space into the GPU's address space, and the GPU threads access the CPU memory space with load/store instructions [33, 37].

Some collective communication primitives, e.g., `ALLREDUCE`, require computation. In such primitives, computation is often pipelined to multiple stages along with communication for better performance. If the CPU-initiated DMA transfer is used, synchronization between the CPU and the GPU is necessary for each stage. Thus, collective communication libraries for GPUs like NCCL prefer the GPU-initiated transfer to finish the collective communication with a single GPU kernel launch.

GPUs can also let other devices access their memory space directly. In PCIe, each device can expose its memory space to the PCIe memory address space with a Base Address Register (BAR). Then, the devices, such as NICs and other GPUs, can directly access the GPU memory space by sending PCIe memory transactions. Different GPU vendors have different names for such a feature, e.g., NVIDIA's *GPUDirect* [43] and AMD's *DirectGMA* [14]. However, not all GPUs expose their internal memory space to the PCIe, and only high-end GPUs support the feature. When a GPU does not support such direct access, the GPU and PCIe devices should communicate via a bounce buffer in the CPU's main memory. In other words, the sender GPU first writes data on the CPU memory, and then the receiver GPU reads data from the CPU memory. In some systems, GPUs have dedicated high-speed interconnects, such as NVLink [44] and xGMI [15], to enable direct access from other GPUs.

NUMA architectures. In a Non-uniform Memory Access (NUMA) architecture, a specific set of memory channels and CPU cores forms a NUMA node. Accessing the memory attached to a remote NUMA node is usually slower than accessing the memory in a local NUMA node because the access passes through the CPU-to-CPU interconnect, such as Intel's QPI [19] or AMD's Infinity Fabric [15].

A single CPU is often configured as multiple NUMA nodes. For example, AMD's EPYC CPU consists of multiple SoC dies, and each die becomes a NUMA node [54]. Another example

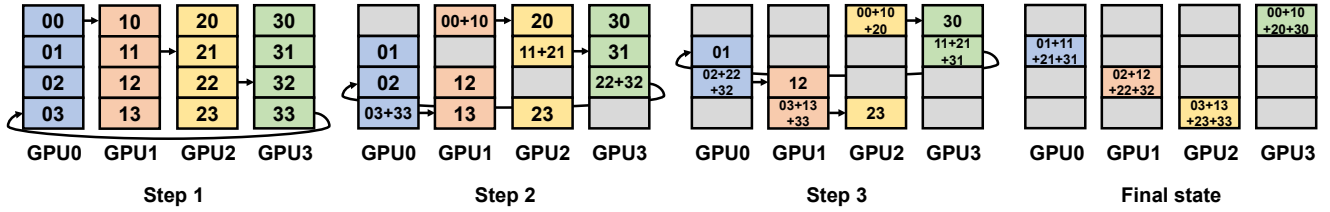


Figure 3. Ring algorithm of REDUCESCATTER for four GPUs.

is Intel’s sub-NUMA cluster mode [34] that allows grouping CPU cores and memory channels close to each other.

Like memory channels, PCIe lanes are also connected to a specific NUMA node in the NUMA architecture. Thus, PCIe devices attached to the lanes, such as the GPUs and the NICs, belong to the NUMA node that the lanes are connected to. The access latencies and bandwidths between the GPUs, the NICs, and the CPU main memory depend on whether they belong to the same NUMA node.

2.2 Collective Communication

A collective communication operation exploits a specific pattern in which workers in a group communicate. Many communication libraries, such as OpenMPI [9] and NCCL [40], efficiently implement collective communication primitives. They serve as building blocks for many real-world applications. Numerous algorithms have been designed, such as recursive doubling [60], Rabenseifner’s algorithm [49], double binary tree algorithm [55], and ring algorithm [47].

Ring algorithm. Here, we briefly introduce the ring algorithm on which this paper focuses. Figure 3 shows how a basic ring algorithm works for a REDUCESCATTER across four GPUs. First, each GPU divides the data into N blocks, where N is the number of GPUs participating in the communication. In step j , the i -th GPU sends its $(i + j)$ -th block to the following GPU, receives the block from the previous GPU, and performs reduction. Note that every GPU sends the data block to its following GPU *simultaneously* in each step. Thus, if all transfers between adjacent GPUs can happen simultaneously at a maximum rate, the ring algorithm for REDUCESCATTER will take $\frac{M(N-1)}{B}$ where M is the size of data, and B is the bandwidth of the slowest link between the adjacent GPUs. The ring algorithm similarly works for ALLGATHER, REDUCE, and BROADCAST. For ALLREDUCE, we can execute REDUCESCATTER followed by ALLGATHER to obtain the net result of ALLREDUCE.

Collective communications in DL. The type of collective communication primitives used in DL depends on the parallelism used. In data parallelism, gradients from each data parallel group should be accumulated. Thus, ALLREDUCE is used after the backpropagation of each batch.

Megatron [59] uses tensor parallelism that distributes linear and attention layer parameters across GPUs. Each GPU computes the partial sum of the layer’s output during the forward pass, and ALLREDUCE is used to compute the total sum. Similarly, during the backward pass, each GPU computes the partial sum of the layers’ input gradient, and ALLREDUCE is used to accumulate them.

Sequence parallelism [22, 25, 28] additionally distributes computation along the sequence dimension. For example, Megatron [25] distributes LayerNorm and Dropout layers in the Transformer architecture. Thus, it performs ALLGATHER and REDUCESCATTER before and after each of the two layers.

In ZeRO-DP (or fully sharded data parallelism) [52], the parameters of each layer are distributed across GPUs. Before the forward pass of each layer, parameters should be ALLGATHERed to process the layer. Since the parameters are distributed, there is no need for all the gradients to be shared across all GPUs. Thus, REDUCESCATTER is used to receive only a subset of accumulated gradients corresponding to parameters stored in each GPU instead of ALLREDUCE.

In expert parallelism (Mixture of Experts, or MoE) [8], ALLToALL is used to send activations computed in different GPUs to the corresponding experts.

3 TCCL

In this section, we describe how TCCL works and present interesting congestion patterns found with TCCL. Before proceeding into details, we introduce the notations and define the terms used in the paper. A *transfer* is a data movement from one device to another without going through an intermediate device. A device is either the CPU’s main memory or a GPU. In a cluster of multiple nodes, we denote the j -th GPU in the i -th node as $G_{i,j}$. Similarly, we indicate the CPU’s main memory of the k -th NUMA node in the i -th node of the cluster as $M_{i,k}$. When we mention a single node in the cluster, we omit the node index from the notations.

We denote the transfer from source x to destination y as $x \rightarrow y$. For example, $G_0 \rightarrow G_1$ denotes the direct data transfer from G_0 to G_1 . On the other hand, when G_0 sends data to G_1 via a bounce buffer on the CPU memory, two transfers are involved in the transfer, e.g., $G_0 \rightarrow M_2$ and $M_2 \rightarrow G_1$.

We use a set notation when we refer to multiple transfers running simultaneously. For a given transfer set, suppose that there exists an enumeration $t_1, t_2, \dots, t_n$, where n is the size of the transfer set and t_i is its element. Also, suppose that the destination of t_i matches the source of t_{i+1} for all $1 \leq i < n$. Then, we say the transfer set is a *linear chain*. A chained notation may be used for a linear chain. For example, $\{G_0 \rightarrow M_2, M_2 \rightarrow G_1, G_1 \rightarrow G_2\}$ is a linear chain and can be written as $G_0 \rightarrow M_2 \rightarrow G_1 \rightarrow G_2$. We call the first and last transfers of a linear chain *head* and *tail*, respectively. When the head's source and the tail's destination match, we say that the transfer set is also a *ring*, e.g., $G_0 \rightarrow M_2 \rightarrow G_1 \rightarrow G_0$.

3.1 TCCL Profiler

Many benchmarks, such as OSU Micro-Benchmarks [2], nvbandwidth [41], and Scope [46], exist to measure the latency and bandwidth of transfers involving GPUs and NICs. One might expect that executing multiple instances of such benchmarks using multi-process launchers (e.g., `mpirun`) is enough to measure congestion. However, since each benchmark has an initialization phase with a non-deterministic latency (e.g., due to memory allocation), multiple transfers are not guaranteed to start simultaneously, leading to inconsistent overlap between transfers and resulting in inaccurate measurements.

Besides, since a substantial number of communication paths need to be profiled, spawning a new set of benchmark processes each time causes multiple initialization phases and significant process launch overhead, resulting in slow pathfinding.

To this end, we implement a multi-process profiler specialized in our use case. Figure 4 describes differences between the existing approaches and TCCL's. Instead of spawning processes each time, it maintains a process pool of size equal to the maximum number of processes to execute. Each process initializes underlying libraries and allocates the memory space necessary for any transfer. Each set of transfers is distributed to processes and launched after synchronization between participating processes. We treat inter-node transfers separately; they are handled by two processes, one in a sender node and the other in a receiver node.

For a set of transfers T , the TCCL profiler measures the time for all transfers to finish, where each transfer sends the same N -byte of data simultaneously, mimicking the patterns in the ring algorithm. Each transfer sends large enough data so that the bandwidth is saturated. The TCCL profiler averages the time across multiple iterations after a few warm-up iterations. Then, it reports the final averaged transfer time and the achieved bandwidth, N divided by the transfer time, back to the TCCL pathfinder. We define $B(T)$ as the bandwidth of the transfer set T reported by the TCCL profiler.

3.2 Congestion Patterns

We observe that existing collective communication libraries fail to achieve the performance that their performance model estimates. To pinpoint the root cause, we first identify the communication paths they use in two clusters, AMD-V100 and Intel-V100. The system configurations are described in detail in Table 2, and Figure 5 describes the communication topology of nodes in each cluster. Then, we profile the set of transfers in the paths and check if the bandwidth $B(T)$ is bounded by the transfer with the smallest bandwidth or not, i.e., $B(T) = \min\{B(\{t\}) \mid t \in T\}$. If $B(T)$ equals the smallest bandwidth, the transfers are progressing well in parallel, and the performance is just bounded by the slowest transfer, i.e., no congestion. Otherwise, congestion exists in that some transfers affect others. In the following, we describe interesting congestion patterns we found.

Table 1. Transfer sets profiled.

No.	Cluster	Profiled transfer set	Bandwidth
1	AMD-V100	$\{M_1 \rightarrow G_1, G_1 \rightarrow G_0\}$	4.27 GB/s
2	AMD-V100	$\{M_1 \rightarrow G_1\}$	13.09 GB/s
3	AMD-V100	$\{G_1 \rightarrow G_0\}$	10.86 GB/s
4	AMD-V100	$\{M_1 \rightarrow G_1, G_1 \rightarrow G_2\}$	9.85 GB/s
5	AMD-V100	$\{M_1 \rightarrow G_1, G_1 \rightarrow M_3\}$	11.27 GB/s
6	Intel-V100	$\{G_0 \rightarrow G_2\}$	0.42 GB/s
7	Intel-V100	$\{G_0 \rightarrow G_1\}$	7.45 GB/s
8	Intel-V100	$\{G_0 \rightarrow M_0\}$	13.16 GB/s
9	Intel-V100	$\{G_0 \rightarrow M_0, M_0 \rightarrow G_2\}$	12.29 GB/s
10	AMD-V100	$\{G_0 \rightarrow M_1, M_1 \rightarrow G_1, G_1 \rightarrow M_1, M_1 \rightarrow G_2\}$	8.01 GB/s
11	AMD-V100	$\{G_0 \rightarrow M_1, M_1 \rightarrow G_1, G_1 \rightarrow M_2, M_2 \rightarrow G_2\}$	11.28 GB/s
12	Intel-V100	$\{G_2 \rightarrow M_0, M_0 \rightarrow G_0, G_0 \rightarrow M_0, M_0 \rightarrow G_1\}$	9.65 GB/s
13	Intel-V100	$\{G_2 \rightarrow M_0, M_0 \rightarrow G_0, G_0 \rightarrow M_1, M_1 \rightarrow G_1\}$	11.27 GB/s

CPU memory accesses and GPU direct transfers. On AMD-V100, the most interesting congestion pattern is when a GPU reads the CPU memory, and the same GPU directly writes to the memory of a GPU in a different NUMA node simultaneously. Suppose in Figure 5a, G_1 reads M_1 on the NUMA node 1, and G_1 writes to G_0 that resides on the NUMA node 3. The first transfer $M_1 \rightarrow G_1$ alone achieves 13.09 GB/s and the second transfer $G_1 \rightarrow G_0$ alone achieves 10.86 GB/s, where both are acceptable performance for PCIe Gen3 link. However, if we execute them simultaneously, it only achieves 4.27 GB/s. As both transfers overlap only at the PCIe link between G_1 and CPU, and the link is bidirectional, they should achieve much better performance in theory. Comparably, if a GPU directly writes to the memory of another GPU in the same NUMA node, it achieves a much better bandwidth of 9.85 GB/s. If a GPU writes to the CPU memory in a different NUMA node, it also achieves 11.27 GB/s.

In Intel-V100, a direct transfer between the GPUs in different NUMA nodes shows unexpectedly low performance even if the transfer does not interfere with other computations or communications. Since the bandwidth of the link

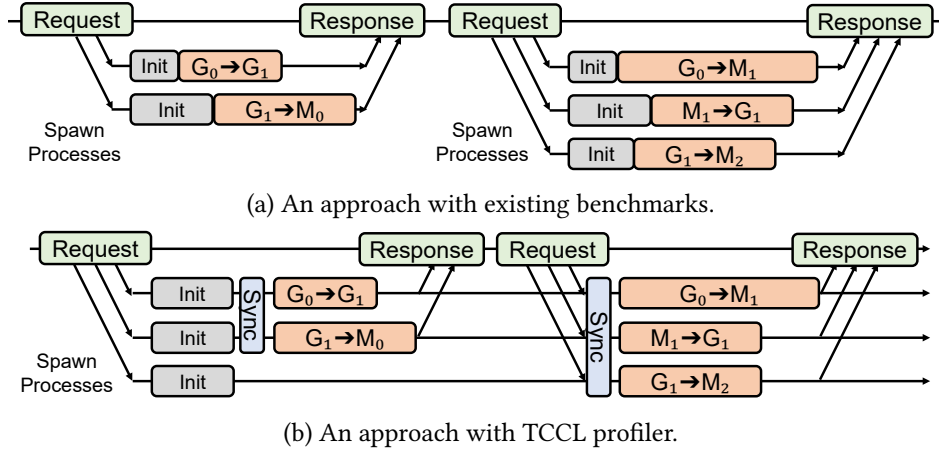


Figure 4. Comparison between the existing approaches and TCCL's approach. The pathfinder requests two sets of transfers: $\{G_0 \rightarrow G_1, G_1 \rightarrow M_0\}$ and $\{G_0 \rightarrow M_1, M_1 \rightarrow G_1, G_1 \rightarrow M_2\}$. Note that the time scale in the figure is not exact.

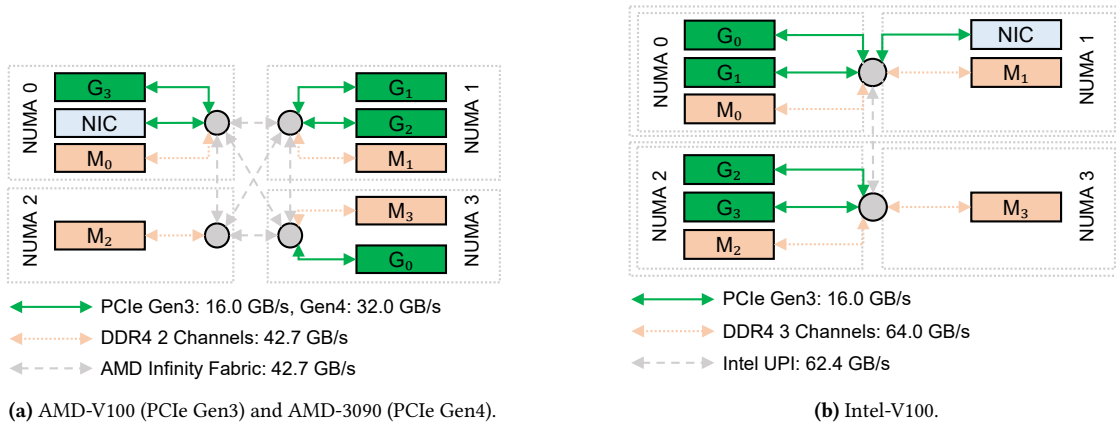


Figure 5. Topology and link bandwidth of a node from clusters. DDR links are half-duplex, while other links are full-duplex.

between NUMA nodes, in this case Intel UPI, is larger than the bandwidth of the PCIe link, it should not be a bottleneck in theory. The direct transfer between the GPUs in the same NUMA node or the transfer to the CPU memory does not show such performance degradation. Thus, the transfer via the bounce buffer in the CPU memory is beneficial when crossing NUMA boundaries in Intel-V100. It only utilizes the transfer to/from the CPU memory and does not involve a direct transfer between the GPUs through the CPU interconnect.

Table 1 shows the profiled bandwidth of the transfers mentioned above. Set 1 describes the congestion between the CPU memory access and the access to the GPU memory in a different NUMA node. Sets 2 and 3 show that each transfer alone achieves much better performance, and Sets 4 and 5 show that altering the destination alleviates congestion. Set 6 shows poor performance of a direct transfer between GPUs

through the CPU interconnect. If a source or a destination is not a GPU, like Set 7 and 8, it achieves better bandwidth even though the transfers are still through the CPU interconnect. Set 9 shows the benefit of a transfer via the bounce buffer (here, M_0) instead of the direct transfer like Set 6.

Between CPU memory accesses. Since the CPU memory bandwidth is limited, performance may degrade when too many transfers are concentrated on the CPU memory in the same NUMA node. Set 10 in Table 1 shows the simultaneous transfers when the GPUs G_0 , G_1 , and G_2 on AMD-V100 communicate via the bounce buffer on M_1 . All four transfers access the same CPU memory, so bandwidth degrades to 8.01GB/s. However, if one of the buffers are allocated on M_2 , the bandwidth improves by 40% (Set 11). Set 12 and Set 13 show the same phenomenon on the Intel-V100 cluster.

3.3 TCCL Pathfinder

Since the number of possible communication paths increases exponentially with the number of GPUs in a cluster, it is infeasible to try all communication paths in the cluster. We introduce the algorithm of the TCCL pathfinder to find performant paths in a feasible amount of time.

3.3.1 Intra-node Pathfinding. It is trivial that a set of simultaneous transfers will achieve lower bandwidths as more transfers come into play because the congestion can only be more severe, not disappear. This can be described as:

$$T_1 \subset T_2 \Rightarrow B(T_1) \geq B(T_2), \quad (1)$$

where T_1 and T_2 are sets of simultaneous transfers. Due to the above property, we can find the best ring inside a node analogous to Dijkstra's shortest path algorithm [7].

Algorithm 1: Intra-node Pathfinding algorithm

```

1 Function FindBestIntraNodeRing( $\mathbb{G}$ ):
2    $Q \leftarrow \text{PriorityQueue}()$ 
3    $Q.\text{insert}(\text{key} = \infty, \text{value} = \{\})$ 
4    $C \leftarrow \text{Dictionary}()$  /* Cache */
5   while  $Q$  is not empty do
6      $T, BW \leftarrow Q.\text{pop}()$ 
7     if  $T$  forms a ring then
8       return  $T, BW$ 
9     end
10     $T_{\text{cand}} \leftarrow \text{GetCandidates}(T, \mathbb{G})$ 
11    for  $t_{\text{new}} \in T_{\text{cand}}$  do
12       $T_{\text{new}} \leftarrow T \cup \{t_{\text{new}}\}$ 
13      if  $T_{\text{new}}$  in  $C$  then
14         $BW_{\text{new}} \leftarrow C(T_{\text{new}})$ 
15      end
16      else
17         $BW_{\text{new}} \leftarrow \text{Profile}(T_{\text{new}})$ 
18         $C.\text{insert}(\text{key} = T_{\text{new}}, \text{value} = BW_{\text{new}})$ 
19      end
20       $Q.\text{insert}(\text{key} = BW_{\text{new}}, \text{value} = T_{\text{new}})$ 
21    end
22  end

```

Algorithm 1 describes the method to find the best ring inside a node. The algorithm starts with a set of GPUs to visit $\mathbb{G}$ and a maximum priority queue Q that uses the bandwidth as a key and the transfer set as a value. The queue will contain all transfer sets currently profiled but not yet explored further by appending more transfers, similar to the priority queue-based implementation of Dijkstra's algorithm.

The queue is first initialized with an empty transfer set. At each iteration, TCCL picks the currently best transfer set T from the queue and tries to append all available transfers that advance T one step further to form a ring. For example, if $\mathbb{G}$ is $\{G_0, G_1, G_2\}$ and T is $G_0 \rightarrow G_1 \rightarrow G_2$, then $G_2 \rightarrow G_0$ can

be added to make T a ring. Alternatively, $G_2 \rightarrow M_i$ can be added to use a bounce buffer in a NUMA node i and let the future iteration add $M_i \rightarrow G_0$ to finish a ring. However, adding transfers such as $G_2 \rightarrow G_1$ will make the set unable to form a ring. The routine `GetCandidates` returns appropriate transfers to append by considering the current transfers and unvisited GPUs. The new transfer set is sent to the TCCL profiler, and the result is inserted back into the priority queue. TCCL also maintains a cache to prevent a transfer set from being profiled again unnecessarily.

If the current transfer set T is a ring, it is guaranteed that T achieves the best bandwidth. This is because all other sets in the priority queue have less bandwidth by the definition of the maximum priority queue, and unexplored sets will have a less bandwidth due to the property 1. We execute the algorithm and save the best transfer set as $DB_{\text{intra}, \mathbb{G}}$ for each subset $\mathbb{G}$ of GPUs in a node to support the cases where only a subset of GPUs participates in collective communication.

Exploiting NUMA symmetry. We exploit the symmetry of NUMA architecture to reduce the search space further. Let T_1 and T_2 be transfer sets and N be the number of NUMA nodes. If there exists a bijection $f: \mathbb{Z}_N \rightarrow \mathbb{Z}_N$ that can map devices on NUMA node i in T_1 to devices on NUMA node $f(i)$ in T_2 , we say T_1 and T_2 are equivalent up to the NUMA symmetry and, in turn, their bandwidths are the same. By considering all equivalent transfer sets up to the NUMA symmetry as a cache hit, TCCL can dramatically reduce the number of transfer sets to profile.

3.3.2 Inter-node Pathfinding. The algorithm for the intra-node is not directly applicable to the inter-node pathfinding because the number of different paths that travel through all GPUs in a cluster becomes extremely large. Instead, we take an alternative approach to construct short linear chains that visit GPUs in a single node, merge them into a linear chain that visits all GPUs in a cluster, and join both ends of the linear chain to form a ring. We assume that all nodes in the cluster have identical architecture and are indexed from 1 to N .

When we first make a linear chain T_n that visits GPUs in the n -th node, we consider the transfers whose head is the inter-node transfer from the previous node and its tail is the inter-node transfer to the next node. Here, an inter-node transfer from node i to node $i + 1$ is one of four types: $G_{i,j} \rightarrow G_{i+1,k}$, $G_{i,j} \rightarrow M_{i+1,k}$, $M_{i,j} \rightarrow G_{i+1,k}$, and $M_{i,j} \rightarrow M_{i+1,k}$ where j and k are the index of a GPU or an NUMA node. When the source or the destination is a GPU, the NIC will directly write to or read from the GPU memory.

Then, we merge two linear chains T_n and T_{n+1} that visit GPUs in n -th node and $(n + 1)$ -th node, respectively. Figure 6 describes an example of such merging. The tail of T_n and the head of T_{n+1} should be the same transfer because $T_n \cup T_{n+1}$ will not be a linear chain otherwise. The question is how to

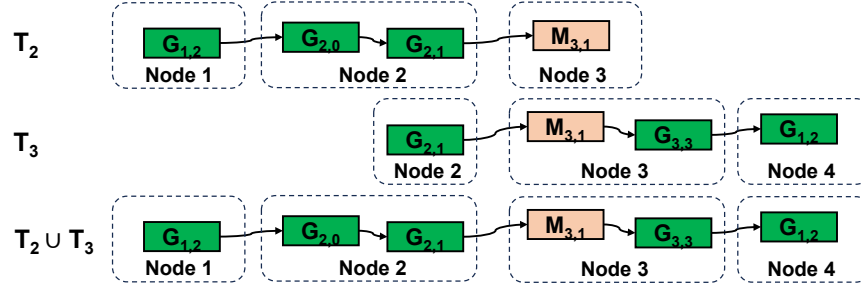


Figure 6. An example of merging two linear chains. Note that the tail of T_2 and the head of T_3 is the same inter-node transfer, $G_{2,1} \rightarrow M_{3,1}$, so they can be merged.

estimate $B(T_n \cup T_{n+1})$ without profiling the new linear chain after every merge. By the property described by Equation 1, the bandwidths of T_n and T_{n+1} are the upper bound to the bandwidth of the new chain:

$$B(T_n \cup T_{n+1}) \leq \min(B(T_n), B(T_{n+1})).$$

Our observation is that performance degradation in PCIe-dependent systems largely stems from the congestion occurring inside nodes. As T_n contains all transfers that pass the n -th node, including the inter-node transfers, all congestion inside a node are already incorporated in $B(T_n)$. Assuming there is no congestion at the inter-node network, we use the minimum bandwidth of two linear chains to estimate the bandwidth of the merged linear chain, i.e., we let $B(T_n \cup T_{n+1})$ to be $\min(B(T_n), B(T_{n+1}))$.

We use dynamic programming to find the linear chain with the best bandwidth from merging. The sub-problem for dynamic programming, $D[n][h][t]$, is defined as the best bandwidth of a linear chain that visits GPUs participating in collective communication from the first node to the n -th node. h is the head of the linear chain, and t is the tail of the linear chain.

As a base case, we first find the best linear chain for n -th node as $DB_{\text{inter}, \mathbb{G}_n}[h][t]$, where its head is h and its tail is t . $\mathbb{G}_n$ is the set of GPUs participating collective communication in the n -th node. We can find the best linear chain with Algorithm 1, with a small modification to GetCandidates and changing the end condition (Line 7) to a linear chain. Similar to intra-node pathfinding, $DB_{\text{inter}, \mathbb{G}_n}$ is pre-computed for every possible subset $\mathbb{G}_n$. Then, we compute the best bandwidth of the merged linear chain. This can be solved iteratively with the following formula:

$$D[n][h][t] = \max_x (\min(D[n-1][h][x], DB_{\text{inter}, \mathbb{G}_n}[x][t])), \quad (2)$$

where x is an inter-node transfer. The algorithm merges the best $(n-1)$ -node linear chain that ends with x and the best single-node linear chain that starts with x , and saves the best result after trying all available inter-node transfers.

As a final step, the best ring can be found among linear chains where its head and tail can be joined to form a ring

as follows:

$$R = \max_x D[N][x][x], \quad (3)$$

where N is the number of nodes, and R is the bandwidth for the final ring.

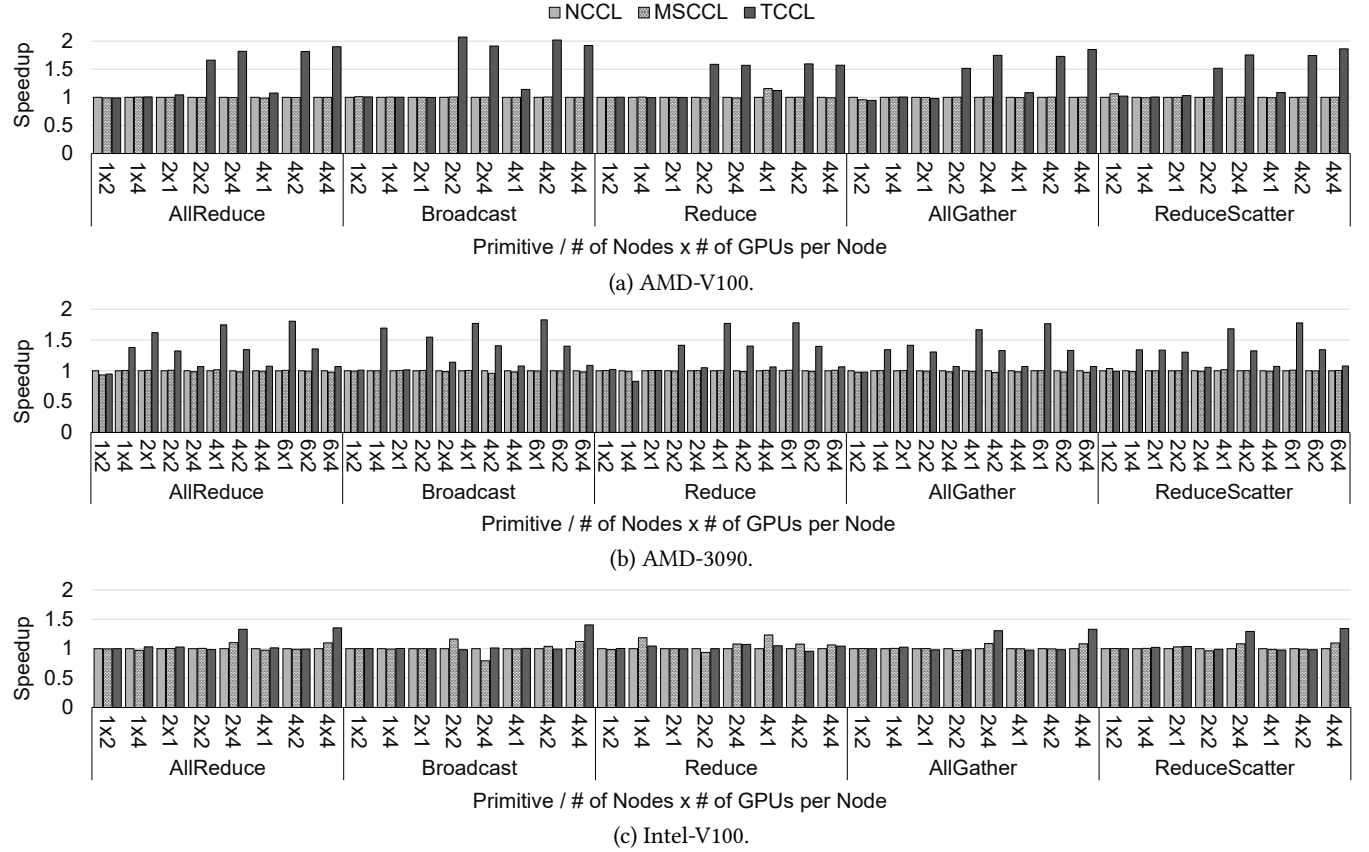
3.4 Integration to NCCL

Instead of implementing the whole collective communication library from scratch, we implement TCCL runtime on top of NCCL. Users need to execute the TCCL pathfinder only once when they install TCCL, and its result is fed into the TCCL runtime as an XML file. As NCCL already implements the ring algorithm, we reuse GPU kernels of the ring algorithm and corresponding host implementations. However, the TCCL runtime automatically modifies the ring's path and the types of transfers, such as direct P2P transfers and transfers via a bounce buffer.

In NCCL, the set of GPUs participating in collective communication primitives, $\mathbb{G}_n$, is known to the runtime when the user application creates a communicator object. If only the GPUs in a single node participate, we can immediately select the best ring from the intra-node pathfinding, as we save the best transfer set for all subsets of GPUs. When it involves GPUs from multiple nodes, the ring should be searched with the dynamic programming each time the communicator is created, as we cannot store solutions for all combinations of GPUs in the cluster. The size of state space for dynamic programming is $N \times I^2$, where N is the number of nodes and I is the number of available inter-node transfer types. As the source and the destination of inter-node transfer are either G_i and M_k , I is the square of the sum of the number of GPUs and the number of NUMA nodes in a single node. The inter-node pathfinding takes less than a second because the state space size is not large (usually less than a billion). Besides, as the communicator is usually created in the initialization phase of the user application, this does not cause any overhead during the collective communication primitives.

Table 2. Clusters used in the evaluation. The AMD-V100 and AMD-3090 clusters are equipped with ASRock ROMED8-2T mainboards. The Intel-V100 cluster is equipped with Supermicro X11DPG-QT mainboards.

Cluster	Nodes	CPU	Memory	GPU	NIC	PCIe
AMD-V100	4	1 x AMD EPYC 7452	8 x DDR4-2666 64GB	4 x NVIDIA Tesla V100-PCIE	Mellanox ConnectX-5 Infiniband EDR	Gen3
AMD-3090	6			4 x NVIDIA GeForce RTX 3090	Mellanox ConnectX-6 Infiniband HDR	Gen4
Intel-V100	4	2 x Intel Xeon Gold 6230	12 x DDR4-2666 64GB	4 x NVIDIA Tesla V100-PCIE	Mellanox ConnectX-5 Infiniband EDR	Gen3

**Figure 7.** Performance of collective communication primitives. For example, 2x4 denotes a total of eight GPUs from 2 nodes (4 GPUs per node) participating in collective communication.

4 Evaluation

In this section, we evaluate TCCL in three different clusters and how much speedup it achieves when applied to DL workloads.

4.1 Evaluation Environment

System configurations. Table 2 describes three different clusters where we evaluate TCCL: AMD-V100, AMD-3090, and Intel-V100. A node in AMD-V100 has an AMD CPU and four NVIDIA Tesla V100s interfaced with PCIe Gen3. In addition, the node is equipped with a 100Gbps NIC that matches the speed of PCIe Gen3. AMD-3090 is the same system as AMD-V100, except the GPUs and NICs are replaced with PCIe Gen4 devices. Unlike NVIDIA V100, NVIDIA RTX

3090 is a gaming-class GPU and does not support direct P2P transfers. Intel-V100 is similar to AMD-V100 but built with Intel CPUs to ensure that TCCL generalizes to different vendors' CPUs. We also note that the NUMA configurations are not arbitrary (i.e., not configured in favor of TCCL) and are rather determined by the width of the GPU PCIe boards and the cooling requirement of the system.

System settings are a critical factor in the performance of collective communication. We adopt the vendor's guide on GPU workstations for high-performance computing [16, 21]. We turn off virtualization supports, such as SR-IOV and IOMMU, which may interfere with P2P transfers. Each AMD CPU is configured as four NUMA nodes per socket so that we can control data locations in the main memory. Similarly,

Table 3. Breakdown of the paths taken in the specified configurations. Transfers that cause bottlenecks are marked red. The bandwidth is the algorithm bandwidth with AllReduce primitive.

No.	Cluster	GPUs	Library	BW	Paths taken
1	AMD-V100	4x4	NCCL	3.08	... → NIC → $M_3 \rightarrow G_0 \rightarrow G_1 \rightarrow G_2 \rightarrow G_3 \rightarrow M_0 \rightarrow$ NIC → ...
2	AMD-V100	4x4	TCCL	5.85	... → NIC → $G_3 \rightarrow M_2 \rightarrow G_2 \rightarrow M_1 \rightarrow G_1 \rightarrow M_3 \rightarrow G_0 \rightarrow M_0 \rightarrow$ NIC → ...
3	AMD-3090	1x4	NCCL	5.25	$G_0 \rightarrow M_3 \rightarrow G_1 \rightarrow M_1 \rightarrow G_2 \rightarrow M_1 \rightarrow G_3 \rightarrow M_0 \rightarrow G_0$
4	AMD-3090	1x4	TCCL	7.39	$G_0 \rightarrow M_2 \rightarrow G_3 \rightarrow M_0 \rightarrow G_1 \rightarrow M_1 \rightarrow G_2 \rightarrow M_3 \rightarrow G_0$
5	AMD-3090	4x1	NCCL	6.17	... → $NIC \rightarrow M_3 \rightarrow G_0 \rightarrow M_3 \rightarrow$ NIC → ...
6	AMD-3090	4x1	TCCL	10.77	... → NIC → $M_0 \rightarrow G_0 \rightarrow M_3 \rightarrow$ NIC → ...
7	Intel-V100	2x4	NCCL	4.48	... → $NIC \rightarrow M_0 \rightarrow G_0 \rightarrow M_2 \rightarrow G_2 \rightarrow M_2 \rightarrow G_3 \rightarrow M_0 \rightarrow G_1 \rightarrow M_0 \rightarrow$ NIC → ...
8	Intel-V100	2x4	TCCL	5.96	... → NIC → $M_0 \rightarrow G_2 \rightarrow M_3 \rightarrow G_1 \rightarrow M_1 \rightarrow G_3 \rightarrow M_3 \rightarrow G_0 \rightarrow M_0 \rightarrow$ NIC → ...

each Intel CPU is configured as two NUMA nodes per socket using Intel’s sub-NUMA cluster mode [34].

Comparison baselines. We compare TCCL with two baselines: NCCL and MSCCL. NCCL (v2.18.3-1) is a representative collective communication library implemented by NVIDIA. It uses a backtracking algorithm that finds paths with maximum bandwidth on the detected topology. NCCL performs the search at the communicator initialization and does not require any separate profile phase. MSCCL (v0.7.4) is a collective communication library that unifies multiple studies [3, 5, 23, 58] from Microsoft, including TACCL synthesizer. We utilize TACCL to synthesize communication schedules and feed the results to MSCCL runtime. While MSCCL uses its own transfer schedules, MSCCL is built on top of NCCL, so it inherits the types of transport that NCCL determines. For example, if NCCL decides to use P2P transfer between G_0 and G_1 , MSCCL does so.

4.2 Collective Communication Primitives

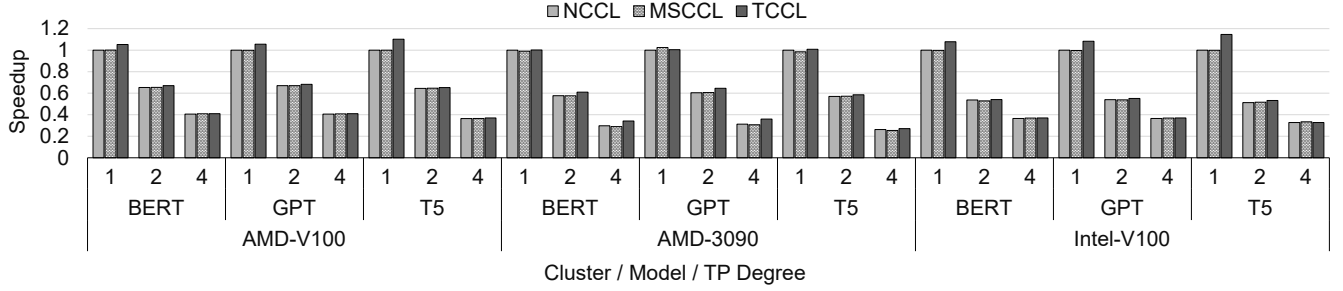
Figure 7 shows the speedup of TCCL over the baselines on each cluster. Table 3 describes the paths the libraries use in detail for a few selected cases. Since the paths from MSCCL are synthesized by solving ILP and are not human-readable, we include only the paths found by NCCL and TCCL. All collective communications are performed with 1GB of data, and the performance is averaged across ten iterations after a single warmup iteration.

Performance on AMD-V100. TCCL shows significant speedup ($\times 1.51$ to $\times 2.06$) in all primitives on multi-node configurations with more than two GPUs. The reason is that NCCL uses the path that includes $NIC \rightarrow M \rightarrow G_0 \rightarrow G_1$, as NCCL decides to use the bounce buffer between the NIC and the GPU and direct transfer between the GPUs (Path 1 in Table 3). However, a pair of transfers, $M \rightarrow G_0$ and $G_0 \rightarrow G_1$, falls into the congestion pattern mentioned in Section 3.2. The communication schedule of MSCCL also includes the same pattern and suffers performance degradation. On the other hand, TCCL captures a path where NIC directly writes to the GPU in the same NUMA and uses the bounce buffer

for other transfers to avoid such congestion patterns (Path 2 in Table 3). For the other cases, all three libraries show comparable performance because all three libraries identify similar communication paths.

Performance on AMD-3090. There is less room to optimize paths in AMD-3090 because its GPUs do not support direct transfers, and all transfers should bounce on the CPU memory. Nevertheless, TCCL achieves speedups up to $\times 1.82$. The speedup mainly comes from the better placement of the bounce buffers on NUMA nodes. NCCL, by default, places the bounce buffer in the NUMA node in which the receiver GPU resides. Thus, in the configurations with four GPUs, the bounce buffers for G_1 and G_2 are both allocated in NUMA node 1 and become the performance bottleneck (Path 3 in Table 3). As MSCCL just utilizes the buffers allocated by NCCL, it suffers from the same bottleneck. However, TCCL distributes the buffers on all four NUMA nodes (Path 4 in Table 3). Similarly, in the cases with a single GPU, NCCL allocates both bounce buffers for NIC-to-GPU and GPU-to-NIC transfer in the same NUMA node as the GPU. Then, the two read transfers and the two write transfers going into the memory in the NUMA node become the performance bottleneck (Path 5 in Table 3). On the other hand, TCCL distributes two buffers to different NUMA nodes, alleviating the bottleneck (Path 6 in Table 3).

In a particular case, REDUCE 1×4 , we find that TCCL is 17% slower than NCCL and MSCCL. This discrepancy arises due to the characteristics of the ring algorithm employed in the REDUCE primitive. Unlike other primitives, the ring algorithm of REDUCE does not fully utilize all links in the ring’s path because the root GPU does not send its data to the successor. For example, if the ring found by the pathfinder is $G_0 \rightarrow G_1 \rightarrow G_2 \rightarrow G_0$, the ring algorithm for REDUCE will only utilize $G_1 \rightarrow G_2 \rightarrow G_0$ if G_0 serves as a root GPU. Since the pathfinder assumes the utilization of all links, this discrepancy between the identified ring by TCCL and the actual optimal path for REDUCE can lead to performance variations. Nonetheless, TCCL exhibits better or comparable performance across other configurations.

**Figure 8.** Speedup in the training of deep learning models.**Table 4.** Configurations of the models used in the training. L is the number of Transformer layers, H is the hidden dimension, n_{heads} is the number of attention heads, and S is the sequence length. A pair of numbers denotes the values applied to the encoder and the decoder, respectively.

Model	L	H	n_{heads}	S
BERT 760M	24	1,536	16	1,024
BERT 1.3B	24	2,064	24	1,024
GPT 760M	24	1,536	16	1,024
GPT 1.3B	24	2,064	24	1,024
T5 760M	12, 12	1,536	16	512, 512
T5 1.3B	12, 12	2,064	24	512, 512

Table 5. Synthesis time taken for TCCL and MSCCL.

Cluster	MSCCL	TCCL		
	Time (s)	Time (s)	Cache hit	Cache miss
AMD-V100	359	27,932	24,942,647	195,653
AMD-3090	8,264	2,937	10,823,989	16,871
Intel-V100	409	8,295	30,237,722	47,298

Performance on Intel-V100. In the cluster with Intel CPUs, direct transfers are generally slower than transfers via bounce buffers. NCCL is aware of this, and its heuristic correctly decides not to use direct transfers. TCCL also finds a path with no direct transfers involved. However, when four GPUs per node are used, TCCL achieves speedups up to $\times 1.40$. Similar to AMD-3090, the placement of bounce buffers by NCCL makes the path visit M_0 three times, resulting in a performance bottleneck. TCCL again avoids this by distributing the buffers to all NUMA nodes.

4.3 Training DL Models

We also evaluate DL training throughput using all GPUs in each cluster when TCCL is used as a communication backend for PyTorch [45], the most widely used DL framework. We evaluate on three different Transformer-based models: GPT [50], BERT [6], and T5 [51]. Table 4 shows the configurations of each model used. As DL training typically requires memory size of at least $\times 16$ model parameters [52], models

are chosen accordingly to fit in the GPU memory. Thus, we run 1.3B-parameter models on AMD-V100 and Intel-V100 and 760M-parameter models on AMD-3090. Global batch sizes of 640, 960, and 640 are used for AMD-V100, Intel-V100, and AMD-3090, respectively. The values are similar to previous studies and large enough to prevent GPU underutilization. We use Megatron [59] as a parallelization framework.

Figure 8 illustrates the speedup achieved across various combinations of clusters, models, and tensor parallelism degrees. When the tensor parallelism degree is 1, which is effectively pure data parallelism, an average speedup of $\times 1.09$ is observed in AMD-V100 and Intel-V100 clusters. The speedup stems from the performance improvement in ALLREDUCE 4×4 as the data parallelism utilizes ALLREDUCE to sum up the gradients from all GPUs before updating the parameters. However, the speedup is small compared to what TCCL achieves in individual primitives because the communication accounts for only around 15% of the total execution time in the data-parallel-only configurations. The speedup in AMD-3090 remains marginal due to the limited speedup of $\times 1.06$ in ALLREDUCE 6×4 .

Tensor parallelism additionally leverages ALLGATHER and REDUCESCATTER $1 \times TP$ to exchange activations within the same tensor parallel group, alongside ALLREDUCE $N \times (4/TP)$ for gradient reduction across tensor parallel groups. While ALLREDUCE only occurs at the end of each training iteration, ALLGATHER and REDUCESCATTER happen before and after each parallel layer, constituting a significant portion of the overall execution time. In AMD-V100 and Intel-V100 clusters, such primitives exhibit minimal speedup, leading to marginal improvements in training with tensor parallelism. In AMD-3090, the speedup achieved by ALLREDUCE 6×1 , ALLGATHER 1×4 , and similar primitives result in $\times 1.05$ and $\times 1.11$ training speedup over NCCL, in tensor parallelism degree of 2 and 4, respectively.

Increasing usability of PCIe-dependent systems. If we employ parallel training algorithms tailored for high-bandwidth systems (e.g., systems with NVLink) in PCIe-dependent systems, GPUs will experience significant underutilization even with the help of TCCL. An example of

such an algorithm is Megatron [35], which recommends exploiting tensor parallelism using all available GPUs within a node. While high-speed interconnects like NVLink can meet the bandwidth requirement of tensor parallelism, Figure 8 shows that employing tensor parallelism is slower than data-parallel-only training in all three PCIe-dependent clusters. Thus, it is essential to explore different combinations of parallelism (e.g., data parallelism and pipeline parallelism) alongside adjusting hyperparameters (e.g., batch size) to reduce the communication-to-computation ratio. As communication volume decreases, other challenges may emerge, such as increased GPU memory requirements and more pipeline bubbles. TCCL can ease this exploration of the design space by independently accelerating communication.

4.4 Time Spent for Path Synthesis

Table 5 summarizes the time the libraries took to find the best paths or schedules. MSCCL requires user to discover the topology and link latency and bandwidth of every link in the system, as well as synthesize schedules for every combination of number of nodes, number of GPUs, and collective primitive to use. We only specify the sum of the times to synthesize schedules for all configurations.

For TCCL, we specify the sum of the times to find the best intra-node rings (i.e., $DB_{intra, \mathbb{G}}$) and the best linear chains (i.e., $DB_{inter, \mathbb{G}}$) for all subsets $\mathbb{G}$ of the GPUs in a node. We observe that the benchmark for each transfer set should run at least 100 milliseconds to get a stable measurement result. Thus, we configured the benchmark to use 32MB as the data size to saturate the bandwidth. It reports bandwidth averaged across ten iterations with a single warmup iteration.

In AMD-V100, TCCL explores about 25 million transfer sets in 8 hours, and 99.2% of them hit the cache. Due to the lack of direct transfers, AMD-3090 has fewer paths to explore and finishes in an hour. For Intel-V100, TCCL has to explore more paths because many transfer sets achieved similar performance, and, in turn, many elements are stored in the priority queue before TCCL finds the final path. However, the NUMA architecture of Intel-V100, where the GPUs are concentrated in two NUMA nodes, makes more transfer sets equivalent up to symmetry and causes a higher cache hit rate than AMD-V100. This causes the search on Intel-V100 to finish much earlier than AMD-V100, although they have the same number of NUMA nodes and GPUs. A few hours of pathfinding is not a big downside as it needs to be executed only once when the system is built.

5 Related Work

In this section, we briefly describe related work on GPU collective communication and performance characterization of PCIe.

5.1 NCCL

NCCL [40] is a representative collective communication library for DL. Using backtracking on the detected system topology, NCCL finds the preferred paths for each predefined communication algorithm (e.g., ring, double binary tree). It uses the theoretical bandwidths adjusted by many experiments to estimate the bandwidth each link can achieve. NCCL then builds a model that estimates the latency from the data size, similar to the well-known Hockney's model [12]. At run time, NCCL chooses an algorithm with the least estimated latency. Since NCCL has been optimized on only the set of systems considered by the NCCL developers, it may achieve sub-optimal performance on the systems not considered by the NCCL developers.

5.2 Synthesis

Some studies do not rely on a well-known algorithm but synthesize new algorithms and find optimized communication paths for a given topology. SCCL [3] encodes the problem as an SMT formula and obtains an exact schedule for Pareto-optimal algorithms. TACCL [58] similarly generates an algorithm by solving an ILP problem with the help of user hints to reduce the search space. GC3 [5] introduces a domain-specific language that compiles user-provided algorithms to GPU communication kernels. Blink [61] considers link heterogeneity to utilize PCIe and NVLink simultaneously.

A limitation of the above approaches is that they often relax some conditions to find a solution in a feasible time, which can sometimes lead to a non-optimal solution. For example, TACCL ignores link congestion in their routing step, which results in sub-optimal performance for a node with only a single NIC.

5.3 Model-Aware Collective Communication

There are many ways to improve DL training throughput by jointly optimizing computation and communication. One is to fuse the communication kernel with the computation kernel, enabling fine-grained control over communication and better computation-communication overlap [23, 48, 62]. Another is to schedule communication and computation differently for a better overlap [30, 56]. P^2 [65] exploits layer dimensions to apply different parallelism, leading to different collective communication primitives executed.

While the model-aware approaches have a higher potential to improve performance, they generally require the source code of GPU kernels which are not available for proprietary libraries, such as cuBLAS [39] and cuDNN [42].

5.4 Accelerating with Specialized Hardware

Some studies improve the performance by utilizing specialized hardware units to offload some workload. SwitchML [57]

Table 6. Previous communication library studies evaluated their proposals with many different systems from different vendors. We denote such a system by the name of the appliance if it is a well-known system. Otherwise, we list the number and the type of GPUs equipped in the system. GPU-to-GPU and GPU-to-NIC columns list the components used when the GPU communicates with another GPU and NIC inside a node, respectively.

Work	System	GPU-to-GPU	GPU-to-NIC
Our work	4× NVIDIA V100, 4× NVIDIA RTX 3090	PCIe host bridge	PCIe host bridge
TACCL [58]	NVIDIA DGX-2, Azure NDv2	NVLink, NVSwitch, PCIe switch	PCIe switch
GC3 [5]	NVIDIA DGX-2	NVLink, NVSwitch	PCIe switch
SCCL [3]	NVIDIA DGX-1, Gigabyte Z52	NVLink, xGMI, PCIe host bridge	-
Blink [61]	NVIDIA DGX-1, NVIDIA DGX-2	NVLink, NVSwitch, PCIe switch	PCIe switch
TACOS [64]	16× NVIDIA V100	NVLink, NVSwitch	PCIe switch
CoCoNet [23]	NVIDIA DGX-2	NVLink, NVSwitch	PCIe switch
Punniyamurthy et al. [48]	4× AMD MI210	xGMI	PCIe host bridge
P^2 [65]	Google Cloud Platform(16× NVIDIA A100)	NVLink, NVSwitch	PCIe switch, PCIe host bridge
SYNDICATE [30]	NVIDIA HGX-2	NVLink, NVSwitch	PCIe switch
C-Cube [56]	NVIDIA DGX-1	NVLink	PCIe switch

and ATP [26] utilize NVIDIA SHARP or programmable network switch to perform reduction on the network switch. PANAMA [10] uses FPGA-based NICs that serve gradient aggregation at the line rate using a lightweight load-balancing protocol. ACE [53] proposes a communication-specialized unit in an NPU to reduce interference with the computation by the communication kernel.

5.5 Target Systems in Previous Studies

Table 6 summarizes the target systems of the previous studies, excluding the specialized hardware approaches. Most studies evaluate their proposals on a high-bandwidth system where communication from/to GPUs does not pass through the CPU. Blink [61] utilizes PCIe links but uses them only as a secondary link to the high-speed links. We observe that previous approaches fail to achieve the expected performance when applied to PCIe-dependent systems where the PCIe link is the main communication link between GPUs.

5.6 Performance Characterization of PCIe

Many studies have analyzed the performance of PCIe links. Neugebauer et al. [36] present a theoretical model for PCIe and benchmarks that characterize the performance of DMA engines and PCIe root complexes across different implementations. Agarwal et al. [1] reveal that contention on the components inside CPUs, such as IOMMU and memory bus, can reduce the bandwidth between CPU and PCIe devices. Li et al. [27] thoroughly evaluate communication between GPUs connected with PCIe and NVLink. These studies focus on a single PCIe link at a time and do not cover cases where multiple PCIe links are used simultaneously. Martinasso et al. [31] model the congestion when the sum of data rates to the specific port on a PCIe switch is higher than the bandwidth of a PCIe link. However, they do not cover congestion patterns mentioned in Section 3.2, which appear even when the sum

of the requested data rates does not exceed the bandwidth of a PCIe link.

6 Conclusion

This paper presents TCCL, a collective communication library for GPU clusters. TCCL targets PCIe-dependent systems, which many researchers rely on but are understudied compared to high-bandwidth systems such as DGX-1. TCCL provides a profiler that measures the bandwidth when multiple transfers proceed simultaneously. Using the profiler, TCCL searches for the best communication path based on the measured bandwidth rather than relying on the theoretical bandwidth. TCCL first finds intra-node paths to reduce the search time and concatenates them to find multi-node paths. TCCL adopts a Dijkstra-like search algorithm and exploits the NUMA symmetry to reduce search space further. Since TCCL is implemented on top of NCCL, it is compatible with existing software using NCCL. The evaluation results of TCCL on three different PCIe-dependent GPU clusters show that TCCL outperforms (up to $\times 2.07$) the state-of-the-art libraries: NCCL and MSCCL for PCIe-dependent systems.

Acknowledgments

This work was supported in part by the National Research Foundation of Korea (NRF) grant (No. RS-2023-00222663, Center for Optimizing Hyperscale AI Models and Platforms), by the Institute for Information and Communications Technology Promotion (IITP) grant (No. 2018-0-00581, CUDA Programming Environment for FPGA Clusters), by the BK21 Plus programs for BK21 FOUR Intelligence Computing (Dept. of Computer Science and Engineering, SNU, No. 4199990214639) and Innovative Data Science Talent Education Program (Dept. of Data Science, SNU, No. 5199990914569) through NRF, all funded by the Ministry of Science and ICT (MSIT) of Korea. ICT at Seoul National University provided research facilities for this study.

A Artifact Appendix

Our artifact contains all source files of TCCL, along with the scripts to reproduce all evaluation results in the paper. It also includes comparison baselines (NCCL and MSCCL) and benchmarks (nccl-tests and Megatron) built for Linux x86-64 systems to simplify installation. This appendix describes how to obtain the artifact, the preprocessing steps, and how to conduct experiments using the scripts provided.

A.1 Evaluation Check List

The following is the check list for artifact evaluation:

- **Algorithm:** Collective communication for GPUs.
- **Program:** We use nccl-tests 2.13.8, PyTorch 2.0.1, and Megatron-LM 23.05 to evaluate TCCL. They are publicly available online.
- **Compilation:** TCCL requires nvcc, NVIDIA CUDA compiler, distributed by NVIDIA.
- **Run-time environment:** TCCL supports Linux, and the provided artifact is tested on Ubuntu 20.04.
- **Hardware:** TCCL targets PCIe-dependent systems, where GPUs are directly attached to the PCIe lanes from CPUs. TCCL currently supports NVIDIA GPUs and Mellanox InfiniBand NICs.
- **Run-time state:** We assume PCIe links and InfiniBand links are dedicated to TCCL.
- **Metrics:**
Bandwidth: bytes transferred in seconds.
Execution time: seconds.
- **Output:** TCCL pathfinder saves the results in XML format. We parse the outputs from the benchmarks and save the bandwidth or the execution time in CSV format.
- **Experiments:** All experiments are executed with shell scripts.
- **How much time is needed to prepare workflow (approximately)?:** About 1 hour to download and build TCCL, comparison baselines, and benchmarks. Communication path synthesis can take up to a few hours; see Table 5.
- **How much time is needed to complete experiments (approximately)?:** About 10 hours to complete experiments for all configurations.
- **Publicly available?:** Yes, on <https://github.com/mcrl/tccl>.
- **Code licenses (if publicly available)?:** BSD license.
- **Archived (provide DOI)?:** <https://dx.doi.org/10.5281/zenodo.10799424>.

A.2 Obtaining TCCL

TCCL can be obtained from GitHub:

```
1 $ wget https://github.com/mcrl/tccl/releases/download/
   tccl-aec/tccl-aec.tar.gz
2 $ tar xf tccl-aec.tar.gz
```

Figure 9 shows the directory structure of the artifact after extracting it.

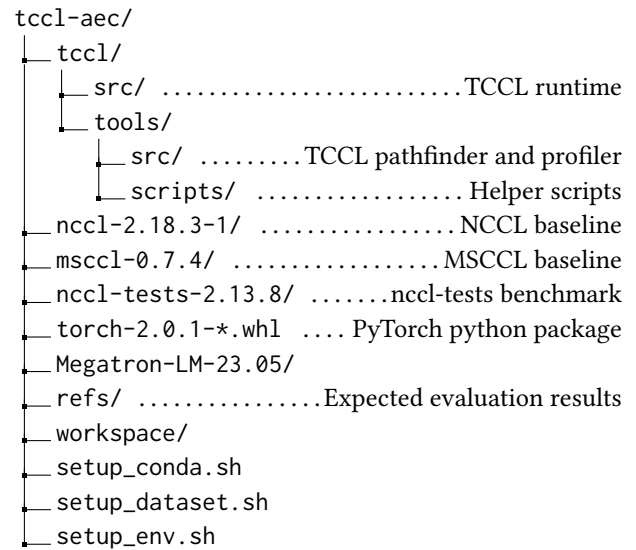


Figure 9. Directory structure of the artifact

Hardware dependences. TCCL focuses on PCIe-dependent systems, where GPUs are directly attached to the PCIe lanes from CPUs. TCCL currently only supports NVIDIA GPUs and Mellanox InfiniBand NICs.

Software dependences. TCCL depends on the NVIDIA CUDA compiler, OpenMPI, libnuma, and libibverbs.

A.3 Installation

First, you should execute initialization scripts as follows:

```
1 $ source tccl-aec/setup_env.sh
2 $ source tccl-aec/setup_conda.sh
3 $ source tccl-aec/setup_dataset.sh
```

The script `setup_env.sh` sets necessary environment variables such as `TCCL_ROOT` used in the later steps. This script should be executed before any other scripts. The script `setup_conda.sh` creates a conda environment and installs PyTorch, Megatron, and their dependencies. The script `setup_dataset.sh` downloads and preprocesses the dataset for deep learning experiments. Both scripts need to be executed only once.

Next, you compile the components of TCCL with GNU make. The following commands will compile both TCCL pathfinder and TCCL runtime:

```
1 $ $TCCL_SCRIPTS/compile_tccl.sh
2 $ ls $TCCL_ROOT/tools/build # pathfinder
3 ... pathfinder ...
4 $ ls $TCCL_ROOT/build/lib # runtime
5 ... libnccl.so ...
```

The comparison baselines and benchmarks are already built and included in the artifact. If you want to build them yourselves, please see <https://github.com/mcrl/tccl/blob/master/AE.md> for detailed instructions.

A.4 Experiment Workflow

Once you have built TCCL, you should execute the pathfinder to generate an XML file that contains communication path information. The pathfinder is an MPI program that runs on multiple nodes, so the launch command depends on the cluster environment. We provide three example scripts that launch the pathfinder on the clusters in Table 2, which assume the SLURM workload manager manages the cluster. Pathfinding may take several hours (Table 5). You will find the XML files in the `tccl-aec/workspace` directory upon completion. `refs/xmls` directory includes XML files as references so you can copy them instead. The scripts in the later steps will assume that XML files are properly placed in the workspace directory.

```
1 $ TCCL_SCRIPTS/launch_pathfinder_amd_v100.sh
2 $ TCCL_SCRIPTS/launch_pathfinder_amd_3090.sh
3 $ TCCL_SCRIPTS/launch_pathfinder_intel_v100.sh
4 (... each script takes a few hours ...)
5 $ ls $TCCL_AEC_ROOT/workspace
6 ... amd_v100.xml amd_3090.xml intel_v100.xml ...
7 # Or copy our results into the workspace
8 $ cp $TCCL_AEC_ROOT/refs/xmls/* $TCCL_AEC_ROOT/workspace
```

You can test TCCL and the generated XML file as follows:

```
1 $ TCCL_SCRIPTS/launch_test_amd_v100.sh
2 ... NCCL INFO TCCL channel setup done ...
3 ... Out of bounds values : 0 OK ...
```

The script will launch 2-node 8-GPU AllReduce with `nccl-tests`. If the installation is successful, the log will state that the TCCL channels have been created properly and that there are no wrong values. You may compare the performance with NCCL by removing `NCCL_ALGO=TCCL` from the scripts.

To use TCCL in applications, replace the shared library of NCCL with that of TCCL using mechanisms such as `LD_PRELOAD`. Note that the shared library of TCCL is named `libnccl.so` to maximize compatibility. You should also set two environment variables: `TCCL_XML_FILE` and `NCCL_ALGO`. `TCCL_XML_FILE` should be the path to the XML file generated by the pathfinder. `NCCL_ALGO` should be set to TCCL to use the communication paths found by TCCL. Otherwise, it will fall back to NCCL's algorithms.

The following is an example of launching an MPI program with TCCL:

```
1 $ mpirun -x LD_PRELOAD=$TCCL_ROOT/build/lib/libnccl.so \
2         -x TCCL_XML_FILE=amd_v100.xml \
3         -x NCCL_ALGO=TCCL \
4         (... other options ...) \
5         ./myapp
```

A.5 Evaluation and Expected results

Collective communication primitives. We provide a script `eval_cc.sh` to reproduce all the data for Figure 7. The script will generate scripts that launch `nccl-tests` with NCCL, MSCCL, and TCCL for all combinations of the clusters, the primitives, the number of nodes, and the number of GPUs used per node. Then, it launches all scripts and stores the outputs into the workspace directory.

```
1 $ TCCL_SCRIPTS/eval_cc.sh
2 $ ls $TCCL_AEC_ROOT/workspace/eval_cc_workspace
3 (... scripts and outputs ...)
4 # Or copy our results into the workspace
5 $ cp -r $TCCL_AEC_ROOT/refs/eval_cc_workspace
   $TCCL_AEC_ROOT/workspace
```

Another script, `organize_cc_result.sh`, will parse the outputs from the `nccl-tests` and organize the measured bandwidths into a single CSV file. We also included our results to compare. You will get the results with an error within 5%.

```
1 $ TCCL_SCRIPTS/organize_cc_result.sh
2 $ ls $TCCL_AEC_ROOT/workspace
3 ... eval_cc_result.csv ...
4 # Compare with our results
5 $ ls $TCCL_AEC_ROOT/refs/*.csv
6 ... eval_cc_result.csv ...
```

Training deep learning models. Similarly, we provide a script `eval_dl.sh` to reproduce all data for Figure 8. The script trains the deep learning models for a few iterations on each cluster with different parallelism.

```
1 $ TCCL_SCRIPTS/eval_dl.sh
2 $ ls $TCCL_AEC_ROOT/workspace/eval_dl_workspace
3 (... scripts and outputs ...)
4 # Or copy our results into the workspace
5 $ cp -r $TCCL_AEC_ROOT/refs/eval_dl_workspace
   $TCCL_AEC_ROOT/workspace
```

Another script, `organize_dl_result.sh`, will extract the execution time per iteration from the logs. We include our results to compare with and expect you to get the execution time with an error within 5%.

```
1 $ TCCL_SCRIPTS/organize_dl_result.sh
2 $ ls $TCCL_AEC_ROOT/workspace
3 ... eval_dl_result.csv ...
4 # Compare with our results
5 $ ls $TCCL_AEC_ROOT/refs/*.csv
6 ... eval_dl_result.csv ...
```

A.6 Customizing Experiments

<https://github.com/mcrl/tccl> includes a more detailed description of the scripts. You can customize the scripts to run experiments on your own cluster and train other deep-learning models.

References

- [1] Saksham Agarwal, Rachit Agarwal, Behnam Montazeri, Masoud Moshref, Khaled Elmeleegy, Luigi Rizzo, Marc Asher de Kruijff, Gautam Kumar, Sylvia Ratnasamy, David Culler, and Amin Vahdat. Understanding host interconnect congestion. In *Proceedings of the 21st ACM Workshop on Hot Topics in Networks*, HotNets '22, page 198–204, New York, NY, USA, 2022. Association for Computing Machinery.
- [2] Devendar Bureddy, Hao Wang, Akshay Venkatesh, Sreeram Potluri, and Dhabaleswar K Panda. OMB-GPU: A micro-benchmark suite for evaluating MPI libraries on GPU clusters. In *Recent Advances in the Message Passing Interface: 19th European MPI Users' Group Meeting, EuroMPI 2012, Vienna, Austria, September 23–26, 2012. Proceedings 19*, pages 110–120. Springer, 2012.
- [3] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. Synthesizing optimal collective algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 62–75, 2021.
- [4] Alibaba Cloud. Alibaba machine learning platform for AI. <https://www.alibabacloud.com/product/machine-learning>, 2023.
- [5] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. GC3: An optimizing compiler for GPU collective communication. *arXiv preprint arXiv:2201.11840*, 2022.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional Transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [7] Edsger W Dijkstra. A note on two problems in connexion with graphs. In *Edsger Wybe Dijkstra: His Life, Work, and Legacy*, pages 287–290. 2022.
- [8] William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1):5232–5270, 2022.
- [9] Edgar Gabriel, Graham E. Fagg, George Bosilca, Thara Angskun, Jack J. Dongarra, Jeffrey M. Squyres, Vishal Sahay, Prabhanjan Kambadur, Brian Barrett, Andrew Lumsdaine, Ralph H. Castain, David J. Daniel, Richard L. Graham, and Timothy S. Woodall. Open MPI: Goals, concept, and design of a next generation MPI implementation. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface: 11th European PVM/MPI Users' Group Meeting Budapest, Hungary, September 19–22, 2004. Proceedings 11*, pages 97–104. Springer, 2004.
- [10] Nadeen Gebara, Manya Ghobadi, and Paolo Costa. In-network aggregation for shared machine learning clusters. *Proceedings of Machine Learning and Systems*, 3:829–844, 2021.
- [11] Google. Google Cloud Vertex AI. <https://cloud.google.com/vertex-ai>, 2023.
- [12] Roger W. Hockney. The communication challenge for MPP: Intel Paragon and Meiko CS-2. *Parallel computing*, 20(3):389–398, 1994.
- [13] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, Hyoungho Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- [14] Advanced Micro Devices, Inc. AMD DirectGMA. https://github.com/GPUOpen-LibrariesAndSDKs/DirectGMA_P2P, 2023.
- [15] Advanced Micro Devices, Inc. AMD Infinity Architecture. <https://www.amd.com/en/technologies/infinity-architecture>, 2023.
- [16] Advanced Micro Devices, Inc. MI200 high-performance computing and tuning guide. <https://docs.amd.com/en/latest/how-to/tuning-guides/mi200.html>, 2023.
- [17] Advanced Micro Devices, Inc. RCCL. <https://github.com/ROCmSoftwarePlatform/rccl>, 2023.
- [18] Amazon Web Services, Inc. Amazon machine learning. <https://docs.aws.amazon.com/machine-learning>, 2023.
- [19] Intel. Intel® QuickPath Interconnect. <https://www.intel.com/content/www/us/en/io/quickpath-technology/quickpath-technology-general.html>, 2009.
- [20] Intel. oneCCL. <https://github.com/oneapi-src/oneCCL>, 2023.
- [21] Intel. Tuning Guides for Intel® Xeon® Scalable Processor-Based Systems. <https://www.intel.com/content/www/us/en/developer/articles/guide/xeon-performance-tuning-and-solution-guides.html>, 2023.
- [22] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Leon Song, Samyam Rajbhandari, and Yuxiong He. DeepSpeed Ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509*, 2023.
- [23] Abhinav Jangda, Jun Huang, Guodong Liu, Amir H. N. Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 402–416, 2022.
- [24] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. Analysis of large-scale multi-tenant GPU clusters for DNN training workloads. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 947–960, 2019.
- [25] Vijay A. Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large Transformer models. *Proceedings of Machine Learning and Systems*, 5, 2023.
- [26] ChonLam Lao, Yanfang Le, Kshiteej Mahajan, Yixi Chen, Wenfei Wu, Aditya Akella, and Michael Swift. ATP: In-network aggregation for multi-tenant learning. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 741–761, 2021.
- [27] Ang Li, Shuaiwen Leon Song, Jieyang Chen, Jiajia Li, Xu Liu, Nathan R Tallent, and Kevin J Barker. Evaluating modern GPU interconnect: PCIe, NVLink, NV-SLI, NVSwitch and GPUDirect. *IEEE Transactions on Parallel and Distributed Systems*, 31(1):94–110, 2019.
- [28] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. Sequence parallelism: Long sequence training from system perspective. *arXiv preprint arXiv:2105.13120*, 2021.
- [29] Liang Luo, Peter West, Jacob Nelson, Arvind Krishnamurthy, and Luis Ceze. PLink: Discovering and exploiting locality for accelerated distributed training on the public cloud. *Proceedings of Machine Learning and Systems*, 2:82–97, 2020.
- [30] Kshiteej Mahajan, Ching-Hsiang Chu, Srinivas Sridharan, and Aditya Akella. Better Together: Jointly optimizing ML collective scheduling and execution planning using SYNDICATE. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 809–824, 2023.
- [31] Maxime Martinasso, Grzegorz Kwasniewski, Sadaf R Alam, Thomas C Schulthess, and Torsten Hoefer. A pcie congestion-aware performance model for densely populated accelerator servers. In *SC'16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 739–749. IEEE, 2016.
- [32] Microsoft. Azure AI. <https://azure.microsoft.com/solutions/ai>, 2023.
- [33] Seung Won Min, Vikram Sharma Mailthody, Zaid Qureshi, Jinjun Xiong, Eiman Ebrahimi, and Wen-mei Hwu. Emogi: Efficient memory-access for out-of-memory graph-traversal in gpus. *arXiv preprint arXiv:2006.06890*, 2020.
- [34] David L. Mulnix. Intel® Xeon® Processor Scalable Family Technical Overview. <https://www.intel.com/content/www/us/en/developer/articles/technical/xeon-processor-scalable-family-technical-overview.html>, 2022.
- [35] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on

- GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2021.
- [36] Rolf Neugebauer, Gianni Antichi, José Fernando Zazo, Yury Audzevich, Sergio López-Buedo, and Andrew W Moore. Understanding pcie performance for end host networking. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pages 327–341, 2018.
- [37] NVIDIA. Peer-to-Peer & Unified Virtual Addressing. https://developer.download.nvidia.com/CUDA/training/cuda_webinars_GPUDirect_uva.pdf, 2011.
- [38] NVIDIA. NVIDIA NVSWITCH - The World's Highest-Bandwidth On-Node Switch. <https://images.nvidia.com/content/pdf/nvswitch-technical-overview.pdf>, 2018.
- [39] NVIDIA. cuBLAS. <https://developer.nvidia.com/cublas>, 2023.
- [40] NVIDIA. NCCL. <https://github.com/NVIDIA/nccl>, 2023.
- [41] NVIDIA. nvbandwidth. <https://github.com/NVIDIA/nvbandwidth>, 2023.
- [42] NVIDIA. NVIDIA cuDNN. <https://developer.nvidia.com/cudnn>, 2023.
- [43] NVIDIA. NVIDIA GPUDirect. <https://developer.nvidia.com/gpudirect>, 2023.
- [44] NVIDIA. NVLink. <https://www.nvidia.com/en-us/design-visualization/nvlink-bridges/>, 2023.
- [45] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, pages 8024–8035, 2019.
- [46] Carl Pearson, Abdul Dakkak, Sarah Hashash, Cheng Li, I-Hsin Chung, Jinjun Xiong, and Wen-Mei Hwu. Evaluating characteristics of cuda communication primitives on high-bandwidth interconnects. In *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*, pages 209–218, 2019.
- [47] Jelena Pjesivac-Grbovic. Towards automatic and adaptive optimizations of MPI collective operations. 2007.
- [48] Kishore Punniyamurthy, Bradford M. Beckmann, and Khaled Hamidouche. GPU-initiated fine-grained overlap of collective communication with computation. *arXiv preprint arXiv:2305.06942*, 2023.
- [49] Rolf Rabenseifner and Jesper L. Träff. More efficient reduction algorithms for non-power-of-two number of processors in message-passing parallel systems. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface: 11th European PVM/MPI Users' Group Meeting Budapest, Hungary, September 19-22, 2004. Proceedings 11*, pages 36–46. Springer, 2004.
- [50] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [51] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text Transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.
- [52] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.
- [53] Saeed Rashidi, Matthew Denton, Srinivas Sridharan, Sudarshan Srinivasan, Amoghavarsha Suresh, Jade Nie, and Tushar Krishna. Enabling compute-communication overlap in distributed deep learning training platforms. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 540–553. IEEE, 2021.
- [54] TIRIAS Research. AMD Optimizes EPYC Memory with NUMA. <https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/AMD-Optimizes-EPYC-Memory-With-NUMA.pdf>, 2018.
- [55] Peter Sanders, Jochen Speck, and Jesper L. Träff. Two-tree algorithms for full bandwidth broadcast, reduction and scan. *Parallel Computing*, 35(12):581–594, 2009.
- [56] Jo Sanghoon, Hyojun Son, and John Kim. Logical/physical topology-aware collective communication in deep learning training. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 56–68. IEEE, 2023.
- [57] Amedeo Sapia, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan Ports, and Peter Richtárik. Scaling distributed machine learning with in-network aggregation. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 785–808, 2021.
- [58] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. TACCL: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 593–612, 2023.
- [59] Mohammad Shoeibi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [60] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. Optimization of collective communication operations in MPICH. *The International Journal of High Performance Computing Applications*, 19(1):49–66, 2005.
- [61] Guanhua Wang, Shivaram Venkataraman, Amar Phanishayee, Nikhil Devanur, Jorgen Thelin, and Ion Stoica. Blink: Fast and generic collectives for distributed ML. *Proceedings of Machine Learning and Systems*, 2:172–186, 2020.
- [62] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, Sameer Kumar, Tongfei Guo, Yuanzhong Xu, and Zongwei Zhou. Overlap communication with dependent computation via decomposition in large deep learning models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 93–106, 2022.
- [63] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 945–960, 2022.
- [64] William Won, Midhilesh Elavazhagan, Sudarshan Srinivasan, Ajaya Durg, Swati Gupta, and Tushar Krishna. TACOS: Topology-aware collective algorithm synthesizer for distributed training. *arXiv preprint arXiv:2304.05301*, 2023.
- [65] Ningning Xie, Tamara Norman, Dominik Grewe, and Dimitrios Vytiniotis. Synthesizing optimal parallelism placement and reduction strategies on hierarchical systems for deep learning. *Proceedings of Machine Learning and Systems*, 4:548–566, 2022.

Received 30 November 2023; accepted 6 March 2024